

AI Audit Report — HW05 Performance Testing

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **Bài:** HW05-AI Performance Testing — **AI Policy:** Open (§9 bắt buộc có phụ lục này)

I use AI tools for the following tasks.

AI tool	Model	Dùng cho
Claude Code	Opus 5 (1M context)	Chọn phạm vi endpoint (§5) · dựng repo + tooling · sinh 4 test plan JMeter · bản mirror k6 · phân tích <code>.jtl</code> · viết báo cáo

Ranh giới cần đọc trước khi chấm mục "human review": các trường *AI sai / bỏ sót* dưới đây ghi lại những gì **phát hiện được trong phiên làm việc** (phần lớn qua smoke test và qua đọc lại code SUT). Trường **Human review** là phần sinh viên tự kiểm và tự chịu trách nhiệm — chỗ nào còn ghi (*sinh viên bổ sung*) là chỗ chưa được sinh viên ký nhận.

Các lượt tương tác

Interaction #1 — Chọn phạm vi endpoint (§5)

- **Task / Scenario:** §5 — chọn endpoint group, chống trùng trong nhóm
- **Bước trong quy trình:** trước `perf-test-plan` bước 1
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 11:05
- **Prompt:** ngoài endpoint nhóm đã chọn thì bạn hãy chọn cho tui 1 vài endpoint đi (kèm 2 ảnh chụp chat nhóm: đăng ký endpoint của Nguyễn, Quan, Thế Đạt; kèm đề bài 2026.HW05.Performance Testing_En.pdf)
- **AI output (tóm tắt):** Đọc 31 route thật trong `backend/server.js`, đối chiếu với đăng ký của 3 thành viên, kết luận toàn bộ luồng khách hàng đã bị chiếm → đề xuất workflow **admin back-office** 5 bước (`login` → `admin/orders` → `admin/users` → `import-products` → `orders/:id/status`) phủ đủ 3 nhóm. Kèm 5 endpoint bổ sung còn trống và 3 lưu ý về SUT (lockout kích hoạt sau 2 lần sai; không có rate limiting; `import-products` báo sai số dòng).
- **AI sai / bỏ sót: Một kết luận sai do chỉ đọc code, không chạy thử.** AI khẳng định `POST /api/admin/import-products` "báo số dòng đã insert nhỏ hơn thực tế" vì `stmt.finalize()` trả response trước các callback `stmt.run` (`server.js:234`). Kiểm bằng request thật ở lượt #8 thì **sai hoàn toàn**: 5/5, 60/60, và 2/3 khi có dòng thiếu `name` — đều đúng, vì `node-sqlite3` xếp các lệnh trên cùng handle theo thứ tự và callback của `finalize` chạy sau chúng. Kết luận này đã lan vào 5 file tài liệu trước khi bị bắt.
- **Vì sao bỏ sót: chất lượng prompt và phương pháp** — prompt hỏi "đọc code và đề xuất endpoint", nên AI trả lời bằng suy luận từ code và trình bày suy luận đó như một sự thật đã kiểm. Không có bước nào bắt nó phân biệt "đọc được từ code" với "đã chạy thử".

- **Human review:** (sinh viên bổ sung — cần xác nhận lại bảng đăng ký của nhóm khớp với chat, và tự đọc `server.js` ở 5 dòng được dẫn để kiểm chứng.)
- **Commit:** docs: playbook, agent skills and report scaffolding

Interaction #2 — Dựng repo và tooling

- **Task / Scenario:** hạ tầng bài làm
- **Bước trong quy trình:** trước `perf-test-plan` bước 1
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 11:10
- **Prompt:** tham khảo các bài hw trc setup repo cho HW05 giùm tui
- **AI output (tóm tắt):** Đọc quy ước HW02/03/04 (`.gitignore`, `README` self-assessment, `tools/`, `.claude/skills/`), dựng repo HW05 cùng bộ khung + 9 script + 4 Agent Skill + 3 tài liệu docs + khung 5 tài liệu §14.
- **AI sai / bỏ sót: Một bug thật trong script do AI viết:** `preflight.mjs` đọc `/usr/libexec/java_home -V` từ `stdout`, nhưng lệnh này in ra `stderr` → script báo `[OK] Java (arm64)` với giá trị **rong**, tức là một lỗi im lặng: báo xanh trong khi không kiểm được gì. Đã sửa bằng hàm `shell()` gộp `2>&1` và đọc `os.arch` từ chính JDK.
- **Vì sao bỏ sót:** đặc điểm của công cụ — hành vi `stdout/stderr` của `java_home` không suy ra được từ ngữ nghĩa lệnh, phải chạy thử mới thấy.
- **Human review:** (sinh viên bổ sung.) Ghi chú kỹ thuật: sau khi sửa, `preflight` phát hiện đúng việc `java` trên PATH là Temurin 8 **x86_64** trong khi máy có Temurin 26 arm64.
- **Commit:** test: environment checks and data-driven fixtures

Interaction #3 — Kế hoạch thực hiện

- **Task / Scenario:** lập kế hoạch
- **Bước trong quy trình:** —
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 11:40
- **Prompt:** giờ tui phải làm j nữa
- **AI output (tóm tắt):** Danh sách 8 bước theo thứ tự ưu tiên theo trọng số điểm (Task 1 = 60/100 nên làm trước), kèm lệnh cụ thể cho từng bước.
- **AI sai / bỏ sót:** —
- **Vì sao bỏ sót:** —
- **Human review:** (sinh viên bổ sung.)
- **Commit:** —

Interaction #4 — Sinh 4 test plan và chạy

- **Task / Scenario:** Load · Stress · Spike · Soak
- **Bước trong quy trình:** `perf-test-plan` bước 1→7
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 12:00
- **Prompt:** hãy thực hiện toàn bộ giùm tui
- **AI output (tóm tắt):** Cài JMeter, seed 50 tài khoản + 430 order + 3 file CSV, viết `tools/gen-test-plans.py` sinh 4 `.jmx` từ **một** định nghĩa workflow dùng chung (§6 đòi 3 plan cùng workflow), chạy

smoke test 20s, sửa lỗi, rồi chạy 3 lượt chính + soak.

- **AI sai / bỏ sót: Hai lỗi thật trong test plan do AI sinh, smoke test mới lộ ra: 1. Nhánh lockout bị tính thành lỗi hiệu năng.** AI gán assertion regex `40[13]` cho request đăng nhập sai mật khẩu và tưởng thế là đủ. Nhưng JMeter mặc định coi mọi `4xx` là Fail, và assertion chỉ **thêm** được lỗi chứ không **xoá** được cờ Fail. Kết quả smoke test: **41,38% error** trong khi SUT hoạt động hoàn toàn đúng. Sửa bằng `JSR223PostProcessor` (Groovy) gọi `prev.setSuccessful(true)` khi mã trả về khớp `40[13]`. 2. **Bước 5 assert cứng HTTP 200, không tính tới state machine.** `PUT /api/admin/orders/:id/status` đi qua FR-10 (`server.js:537-551`): một order chỉ chuyển tiếp được **một lần** cho mỗi trạng thái, nên từ lần lặp thứ hai trở đi SUT trả 400 "invalid transition" — đúng đặc tả. AI còn xen kẽ `shipping` trong `orders.csv`, mà từ `pending` thì `shipping` là chuyển đổi **không hợp lệ** → một nửa sample của bước 5 trả 400 ngay từ đầu. Sửa: `assert 200|400` kèm giải thích, `orders.csv` chỉ dùng `confirmed`, và thêm `tools/reset-orders.mjs` đưa order về `pending` trước mỗi lượt.
- **Vì sao bỏ sót:** cả hai đều thuộc nhóm **đặc điểm của endpoint**, không phải giới hạn model: hành vi "assertion không xoá được cờ Fail" là chi tiết của JMeter chỉ hiện ra khi chạy, và state machine FR-10 nằm trong code SUT chứ không nằm trong đặc tả API. Prompt cũng chưa nêu hai điều này — nên có phần lỗi ở chất lượng prompt.
- **Human review:** (*sinh viên bổ sung*.) Ghi chú: cả hai lỗi đều được phát hiện nhờ **chạy thử 20 giây trước khi chạy lượt 6 phút** — nếu bỏ bước smoke test thì đã có 3 lượt chính với error rate vô nghĩa và phải chạy lại toàn bộ.
- **Commit:** `test(plans): generate the four JMeter plans from one shared workflow`

Interaction #5 — Hai lỗi nữa chỉ lượt chạy dài mới lộ ra

- **Task / Scenario:** Load (lượt chạy thật đầu tiên, đã **hủy giữa lượt**)
- **Bước trong quy trình:** `perf-test-plan` bước 6→7 (chạy và đọc kết quả trước khi tin nó)
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 12:21
- **Prompt:** cùng lượt #4 — đây là phần đọc kết quả và sửa, không phải prompt mới.
- **AI output (tóm tắt):** Lượt Load chạy được, nhưng error rate đứng ở ~2,9% và throughput chỉ ~10 sample/s. Phân tích `.jtl` đang ghi cho thấy hai nguyên nhân độc lập nhau.
- **AI sai / bỏ sót: 3. Dữ liệu test tự đầu độc lượt chạy.** `data/users.csv` do AI sinh có 2 dòng cuối dùng `WrongPassword!` (ý ban đầu là để test lockout). Nhưng thread group chính đọc chính file đó với `recycle=true`, nên nó cũng gặp 2 dòng ấy → login 401 → **hai tài khoản bị khoá 180s** → mọi lần đọc lại 2 dòng đó trả 403, và vì không có token nên **cả 4 bước còn lại của iteration đó cũng 403**. Đó chính là toàn bộ ~2,9% error: đúng 9 lần × 5 label. Không một sample nào trong số đó nói gì về hiệu năng của SUT. Sửa: `users.csv` chỉ còn tài khoản đăng nhập được; nhánh lockout tách sang `data/users_lockout.csv`. 4. **Think-time nhân 5 lần so với ý định.** AI đặt Uniform Random Timer 1000–3000ms ở scope **thread group**, mà JMeter chèn timer trước **từng sampler** — 5 lần một iteration, tức 5–15 giây/iteration thay vì 1–3s. Hệ quả: 20 VU sinh ra ~10 sample/s thay vì ~60, nên "Load test 20 VU" thực chất đo một mức tải nhẹ hơn nhiều lần so với thiết kế. Sửa: chia lại còn 200–600ms mỗi bước để **tổng** mỗi iteration đúng 1–3s, giữ nguyên ngữ nghĩa "admin dừng giữa các màn hình".
- **Vì sao bỏ sót:** lỗi 3 là **chất lượng prompt** — không ai nói rõ "file dữ liệu của luồng chính không được chứa dòng âm tính". Lỗi 4 là **đặc điểm công cụ:** phạm vi tác dụng của timer trong JMeter là

kiến thức phải có sẵn, và AI đã áp dụng sai một cách hoàn toàn im lặng — plan vẫn chạy, vẫn ra số, chỉ là đo sai mức tải.

- **Human review:** (*sinh viên bổ sung.*) Ghi chú: đã **huỷ lượt chạy** giữa đường và **xoá toàn bộ** `.jtl` / **dashboard** / **run-log** của nó thay vì giữ lại để báo cáo. Số liệu từ một lượt đã biết là sai thì không được xuất hiện ở đâu trong bài, kể cả kèm chú thích.
- **Commit:** `test(plans): stop the test data from poisoning its own run`

Interaction #6 — Cùng một lỗi, lần thứ hai

- **Task / Scenario:** Load (lượt chạy thật thứ hai, cũng **huỷ giữa lượt**)
- **Bước trong quy trình:** `perf-test-plan` bước 7
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 13:10
- **Prompt:** cùng lượt #4.
- **AI output (tóm tắt):** Sau khi sửa 4 lỗi trước, lượt Load chạy đúng 50 sample/s như thiết kế và 0% error trong 90 giây đầu. Nhưng error rate rồi leo lên **18,25%**.
- **AI sai / bỏ sót:** 7. **Áp cơ chế đúng cho nhánh lockout mà quên áp cho bước 5.** Toàn bộ 667 sample "lỗi" là `400` của `PUT /api/admin/orders/:id/status` — tức phản hồi **đúng** của state machine FR-10 khi order đã chuyển trạng thái rồi. Assertion `200|400` pass, nhưng JMeter đã gán cờ Fail cho 4xx từ trước và assertion không xoá được cờ đó — **đúng cơ chế đã phát hiện và đã sửa ở lượt #4 cho nhánh lockout**, chỉ là không nghĩ tới việc bước 5 cũng cần. Sửa: dùng lại `mark_expected_4xx("400")` cho bước 5.
- **Vì sao bỏ sót:** không phải giới hạn model cũng không phải đặc điểm endpoint — đây là **suy luận không được mang sang chỗ tương tự**. Đã hiểu đúng cơ chế ở một nơi mà không tự hỏi "chỗ nào khác trong plan cũng trả 4xx hợp lệ?".
- **Human review:** (*sinh viên bổ sung.*) Ghi chú: lỗi này là ví dụ rõ nhất cho việc **error rate là chỉ số phải nghi ngờ trước tiên**. 18% error trên một hệ thống đang hoàn toàn khoẻ — nếu tin con số đó thì báo cáo sẽ kết luận sai hoàn toàn về endpoint transactional.
- **Commit:** `test(plans): count the expected 400 from FR-10 as a pass`

Interaction #7 — Lỗi trong chính tooling đo tài nguyên

- **Task / Scenario:** thu bằng chứng tài nguyên (§6)
- **Bước trong quy trình:** `resource-evidence` bước 2
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 13:19
- **Prompt:** cùng lượt #4.
- **AI output (tóm tắt):** `tools/sample-resources.sh` lấy mẫu CPU/RSS của backend mỗi 2 giây để có số liệu tính được cho endurance threshold.
- **AI sai / bỏ sót:** 8. **Bám vào tiến trình sai.** Script dùng `pgrep -f 'node server.js' | head -1`. Trên máy này `pgrep` khớp **hai** pid: tiến trình `/bin/zsh -c ...` đã khởi động backend (chuỗi đó nằm trong command line của nó) và tiến trình `node` thật. `head -1` chọn cái shell → file CSV ghi **RSS 0,5 MB và CPU 0,0%** suốt 6 phút, trong khi `node` thật đang ở ~76 MB / ~18% CPU. Sửa: lọc theo token đầu của command line phải là `node` (`ps -Ao pid=,command= | awk '/[n]ode server\.js/&& $2 ~ /node$/ ...'`).

- **Vì sao bỏ sót: đặc điểm môi trường.** `pgrep -f` khớp trên toàn bộ command line, nên một shell wrapper chứa cùng chuỗi cũng khớp — điều này chỉ hiện ra khi backend được khởi động qua wrapper, đúng như cách phiên làm việc này khởi động nó.
- **Human review:** (*sinh viên bổ sung.*) Ghi chú: file tài nguyên của lượt **Load** và **Stress** đầu tiên vì thế **vô giá trị ở các cột `node_*`** (cột `java_*` của JMeter vẫn đúng). Đã chạy lại hai lượt đó sau khi sửa, thay thế toàn bộ `.jtl` / `dashboard/resources` — chứ không giữ lại kèm chú thích "cột này không dùng được".
- **Commit:** `test: sample CPU and RSS through each run`

Interaction #8 — Kiểm bug bằng request thật, và một kết luận bị bác bỏ

- **Task / Scenario:** bug report
- **Bước trong quy trình:** `jtl-analysis` bước 3 (phân loại phát hiện: kiểm được hay không)
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 13:58
- **Prompt:** cùng lượt #4.
- **AI output (tóm tắt):** Viết `bug-report/verify-bugs.sh` chạy lại từng bằng chứng, rồi thực thi trên SUT đang chạy.
- **AI sai / bỏ sót: 9. Một "bug" AI khẳng định từ lượt #1 bị bác bỏ khi chạy thử.** AI đã nói `POST /api/admin/import-products` "báo số dòng đã insert nhỏ hơn thực tế" vì `stmt.finalize()` trả response trước các callback `stmt.run` ([server.js:234](#)). Kiểm bằng ba lô request thật: 5/5, 60/60, và 2/3 khi có một dòng thiếu `name` — **cả ba đều đúng.** `node-sqlite3` xếp các lệnh trên cùng handle theo thứ tự và callback của `finalize` chạy sau chúng. Kết luận sai này đã lan vào **5 file** (bug report, endpoint-selection, PLAYBOOK, SKILL perf-test-plan, chú thích trong `gen-test-plans.py` và `k6/workflow.js`) trước khi bị bắt.
- **Vì sao bỏ sót: phương pháp,** không phải model. Suy luận từ code nghe rất hợp lý và có kèm số dòng cụ thể, nên nó được ghi lại như một sự thật đã kiểm. Không có bước nào bắt phải phân biệt "đọc được từ code" với "đã chạy và thấy".
- **Human review:** (*sinh viên bổ sung.*) Ghi chú: bug report giữ lại mục này ở phần **"Ứng viên đã LOẠI"** kèm bảng ba lô kiểm chứng, chứ không xóa đi. Một nhận định sai đã đi vào tài liệu thì việc ghi lại vì sao nó sai có giá trị hơn là làm như chưa từng có nó.
- **Commit:** `docs(bug): verify each finding, and retract the one that failed verification`

Tổng hợp — AI sai gì trên toàn bài

#	Lư ợt	AI sai / bỏ sót	Nhóm lý do (§6)	Đã sửa thành
1	#2	<code>preflight.mjs</code> đọc <code>java_home -V</code> từ stdout trong khi lệnh in ra stderr → báo <code>[OK]</code> với giá trị rỗng (lỗi im lặng)	đặc điểm công cụ	hàm <code>shell()</code> gộp <code>2>&1</code> , đọc <code>os.arch</code> trực tiếp từ JDK
2	#4	Nhánh <code>lockout</code> : <code>assertion regex</code> đúng nhưng không xóa được cờ <code>Fail</code> của <code>4xx</code> → smoke test báo 41,38% error trong khi SUT chạy đúng	đặc điểm endpoint	<code>JSR223PostProcessor</code> gọi <code>prev.setSuccessful(true)</code> cho <code>40[13]</code>
3	#4	Bước 5 <code>assert</code> cứng 200, bỏ qua state machine <code>FR-10</code>	đặc điểm endpoint	<code>assert 200 400 + reset-orders.mjs + orders.csv</code> chỉ dùng <code>confirmed</code>

#	Lượt	AI sai / bỏ sót	Nhóm lý do (§6)	Đã sửa thành
4	#4	<code>orders.csv</code> xen kẽ <code>shipping</code> — chuyển đổi không hợp lệ từ <code>pending</code>	đặc điểm endpoint	chỉ sinh <code>confirmed</code> , sửa cả trong <code>seed-perf-data.mjs</code>
5	#5	<code>users.csv</code> chứa 2 dòng mật khẩu sai → luồng chính tự khoá tài khoản của mình, ~2,9% iteration vô giá trị	chất lượng prompt	tách <code>data/users_lockout.csv</code> , <code>users.csv</code> chỉ còn tài khoản hợp lệ
6	#5	Timer ở scope thread group → think-time chèn 5 lần/iteration, tải thực tế nhẹ hơn thiết kế ~5 lần	đặc điểm công cụ	chia lại 200–600ms mỗi bước để tổng đúng 1–3s
7	#6	Bước 5: 400 hợp lệ của FR-10 vẫn bị tính là lỗi → báo 18,25% error trên hệ thống khoẻ. Cùng cơ chế đã sửa ở lỗi #2 nhưng không mang sang	suy luận không mở rộng	dùng lại <code>mark_expected_4xx("400")</code> cho bước 5
8	#7	<code>sample-resources.sh</code> bám vào tiến trình <code>zsh</code> bao ngoài thay vì <code>node</code> → ghi RSS 0,5MB / CPU 0% suốt lượt chạy	đặc điểm môi trường	lọc theo token đầu command line phải là <code>node</code>
9	#8	Khẳng định <code>import-products</code> báo sai số dòng insert — bác bỏ khi chạy thử (5/5, 60/60, 2/3 đều đúng). Đã lan vào 5 file	phương pháp: suy luận từ code trình bày như sự thật đã kiểm	giữ lại ở mục "đã loại" kèm bảng kiểm chứng; sửa lý do không assert theo <code>inserted</code>
10	#8	<code>Math.min(...array)</code> trong <code>summarize-jtl.mjs</code> tràn call stack với <code>.jtl</code> 264k sample — chạy qua bình thường ở lượt Load 16k	đặc điểm dữ liệu (chỉ lộ ở file lớn)	thay bằng <code>reduce</code>

Nhận xét xuyên suốt: 7 trong 10 lỗi **không làm test plan báo lỗi** — plan vẫn chạy, vẫn ra `.jtl`, vẫn sinh dashboard đẹp. Chúng chỉ làm con số **sai**. Đó là lý do bước "đọc kết quả trước khi tin nó" trong skill `perf-test-plan` không phải thủ tục hình thức: nếu chỉ kiểm "test có chạy không" thì cả 7 lỗi này đều lọt.

Chi phí thật của 10 lỗi: hai lượt chạy phải huỷ và xoá bỏ toàn bộ bằng chứng, cộng khoảng 25 phút chạy lại. Nếu không đọc `.jtl` giữa lượt mà chờ đến khi viết báo cáo mới đọc, thì cái phải làm lại là **cả bốn lượt cộng phần phân tích**.

Bảng này chép sang `report/main-report.md §2.4` — viết một lần, dùng hai chỗ.